Near scale-to-zero on OKE: keep one warm pod, make it tiny

Research report, 27 October 2026, revision 2. Portuguese version: report-pt-BR-dark.pdf.

Scope: how to get most of the savings of scale-to-zero on Oracle Kubernetes Engine (OKE) without paying the cold-start penalty. It covers vertical scaling of the remaining pod, the OKE and Kubernetes features that already help, whether cutting CPU or memory saves more, and how the answer changes for Node.js, PHP and Go.

All manifests referenced here are in `examples/`. The behaviour claims marked **(lab)** were checked on a local Kubernetes 1.36.4 cluster with `lab/run-lab.sh` and `lab/run-runtime-lab.sh`. Cost figures come from `lab/cost-model.py` and OCI's public price list.

Units: CPU and memory amounts use Kubernetes notation. `50m` means 50 millicores, 5% of one CPU; `512Mi` means 512 mebibytes, half a gibibyte. Appendix C explains every unit and shows step by step how the monthly costs are calculated.

Changes in revision 2, after the team's answers:

- Runtime guidance for Node.js, PHP and Go, with lab measurements (section 7).
- Pod density per node, which limits packing once pods are tiny (section 9.5).
- Handling unpredictable traffic, with a query that checks the office-hours assumption (section 10).
- What can go to zero, and what still costs money at zero (section 11).
- OKE cluster variants compared: basic or enhanced, node types, node autoscalers, pod networking, Kubernetes version (section 12).
- New examples 11a to 15. Example 05 fixed: it failed on pods with more than one container.

Added on 28 October: Appendix C, a glossary of the units used for CPU, memory and prices, with a worked cost example.

Contents

1. Summary ... 4
2. Context and assumptions ... 5
3. How the money flows on OKE ... 5
4. What OKE and Kubernetes already provide ... 6
5. Vertical scaling on OKE ... 7
5.1 In-place pod resize: the mechanism ... 7
5.2 Vertical Pod Autoscaler ... 8
5.3 CPU startup boost: small steady state, fast start ... 9
5.4 Shrinking the warm pod during idle periods ... 9
5.5 Interaction rules ... 10
6. CPU or memory: which reduction saves more? ... 10
6.1 Price per unit ... 10
6.2 Where the money goes in one pod (E5 Flex, 24x7) ... 10
6.3 What happens when each one is too low ... 11
6.4 Idle behaviour is different ... 11
6.5 Scenario comparison (30 services, E5 Flex) ... 12
6.6 Verdict ... 12
7. Runtime guidance: Node.js, PHP and Go ... 12
7.1 Node.js ... 13
7.2 PHP ... 13
7.3 Go ... 14
7.4 Cold start: what the lab numbers mean ... 14
8. Horizontal layer: never zero, rarely more than needed ... 15
9. Node layer: turning smaller requests into a smaller bill ... 15
9.1 Cluster Autoscaler (OKE add-on) ... 15
9.2 Karpenter Provider for OCI ... 16
9.3 Consolidation means cold starts ... 16
9.4 Node price levers for the warm pod (50m / 1Gi, 24x7) ... 16
9.5 Pod density: when IP addresses, not CPU or memory, limit packing ... 17
10. Unpredictable traffic ... 17
10.1 Make a wrong guess cheap ... 17
10.2 Spare capacity with placeholder pods ... 18
10.3 Check the office-hours assumption ... 18
10.4 What to do with the result ... 18
11. What can go to zero ... 19
11.1 What "zero" means ... 19
11.2 Candidates ... 19
11.3 What keeps costing money at zero ... 20
11.4 Getting the per-environment figures ... 20
12. Choosing the OKE cluster variant ... 20
12.1 Basic or enhanced cluster ... 20
12.2 Node types ... 21
12.3 Node autoscaler ... 21
12.4 Pod networking (CNI) ... 22
12.5 Kubernetes version ... 22
12.6 If you get to choose ... 22
13. Suggested rollout ... 23
14. Risks and caveats ... 23
15. Open questions for the team ... 24
Appendix A. Files ... 24
Appendix B. Sources ... 25
Appendix C. Glossary of units ... 26
C.1 CPU ... 26
C.2 Memory ... 26
C.3 Prices and time ... 27

C.4 Worked example: what one warm pod costs 27
C.5 Other terms used with numbers 27

1. Summary

1. **Savings come from requests, not usage.** On OKE managed nodes you pay for whole worker nodes. The scheduler places pods by their resource *requests*, and the node autoscaler removes a node only when the requests on it allow it. Shrinking a pod saves money only if smaller requests lead to fewer or cheaper nodes.
2. **One warm, CPU-trimmed pod gets about 90% of the scale-to-zero saving.** In the cost model (30 services, 3 replicas in business hours), true scale-to-zero at night saves 64%. Keeping 1 replica per service with its CPU request cut to 50m saves 58%. The warm pod costs about US$1.40 per service per month (E5 Flex, pay-as-you-go).
3. **Vertical scaling without restarts is ready on OKE.** In-place pod resize is GA since Kubernetes 1.35. OKE currently supports 1.34.10, 1.35.2 and 1.36.4 (1.37.0 in preview). VPA 1.6+ can apply recommendations in place (`InPlaceOrRecreate`), and VPA 1.7+ adds an alpha CPU Startup Boost.
4. **Cut CPU first, memory carefully.**
   - On E5/E6 Flex, 1 vCPU costs the same as 7.5 GB of RAM, so CPU is about 78% of the cost of a typical 500m/1Gi pod.
   - CPU is compressible: a starved container slows down, nobody kills it. Idle CPU usage also really drops to a few millicores.
   - Memory is not compressible. Exceeding the memory *limit* gets the container OOM-killed. Using more than the memory *request* puts the pod first in line for eviction when the node runs short.
   - Whether idle memory drops depends on the runtime. In the lab, PHP-FPM with `pm = ondemand` went from 172 MiB to 10 MiB within 25 s of the last request. Node.js and Go returned burst memory after about 2 and 4.5 minutes. Long-lived caches never drop.
   - Rule: lower CPU requests hard. Lower memory *requests* only down to the measured idle usage. Never lower memory *limits* in place.
5. **Node.js, PHP and Go each read the container limits differently** (section 7):
   - Node.js sets its heap limit from the memory limit once, at start-up (512Mi gives a 259 MiB heap). An in-place resize does not change it, so change memory limits only through a rollout.
   - PHP-FPM with `pm = ondemand` idles at the master process plus OPcache, about 10 MiB in the lab.
   - Go 1.25+ sets GOMAXPROCS from the CPU limit and follows in-place changes within about 1 s, if `go.mod` says `go 1.25` or later. GOMEMLIMIT must be set by hand.
   - Waking a minimal server of any of the three from zero, on a node that already has the image, took 0.8 to 2.1 s. Real applications add their own start-up time, and a new node adds minutes.
6. **Unpredictable traffic is manageable if you shrink requests, not limits** (section 10). A pod with a 25m request and a 1-CPU limit still used 0.91 to 0.99 CPU on a node with free CPU. Scale out on absolute metrics, keep spare node room with placeholder pods, and check the office-hours assumption with the query in section 10.3 before relying on schedules.
7. **"Zero" means zero pods first, then zero nodes** (section 11). Good candidates: non-production environments outside working hours, queue workers in every environment, and rarely used internal tools. At zero you still pay for the control plane of an enhanced cluster (US$73/month), the system node pool (about US$71/month for two small nodes), load balancers beyond the free one, and volumes.
8. **Recommended stack:**
   - KEDA (or HPA) with `minReplicas: 1` on an absolute CPU or request-rate target for user-facing services.
   - VPA in `Off` mode to measure, then `InPlaceOrRecreate` with `RequestsOnly`, or a schedule-based in-place shrinker for off-hours.
   - A CPU startup boost where start-up is CPU-bound.
   - GoKubeDownscaler for whole non-production namespaces, and KEDA with `minReplicaCount: 0` for queue workers.

    - Cluster Autoscaler (OKE add-on) or Karpenter Provider for OCI to turn the freed requests into fewer nodes. Burstable nodes for low-CPU pools. Preemptible nodes only for extra replicas.
9. **Cluster variant, if you get to choose** (section 12): enhanced cluster, Kubernetes 1.36, managed nodes, and the flannel CNI unless you need pod IPs inside the VCN. With VCN-native networking a 2-OCPU node holds at most 31 pods, which for tiny pods costs more than their CPU and memory. The CNI cannot be changed after the cluster is created.
10. **Avoid:**
    - Virtual nodes for tiny warm pods. Allocation rounds up to whole OCPUs and GBs, so a 50m pod costs about 10x more there.
    - HPA on CPU *Utilization* together with shrinking requests. Utilization is usage divided by request, so shrinking the request causes a scale-out storm.
    - Two controllers changing the same pod's resources.
    - Putting the only warm replica on preemptible capacity.

2. Context and assumptions

- **Team context (answers of 27 Oct 2026):**
  - Runtimes: mostly Node.js, plus PHP and Go. The platform should support whatever users need.
  - Traffic: unpredictable for now. Spikes are expected in office hours, but that is not confirmed.
  - No cost split per environment yet.
  - Cluster type, Kubernetes version and node autoscaler are not decided.
- **OKE versions (Oct 2026):** 1.34.10, 1.35.2, 1.36.4, plus 1.37.0 as a preview (OKE release notes). In-place resize needs 1.33+ and is GA from 1.35. Kubernetes 1.35+ requires cgroup v2, which OKE OL8 images provide (OKE 1.35 blog).
- **Cluster type:** I assume *enhanced* clusters (US$0.10/h per cluster). The managed add-ons (Cluster Autoscaler, Metrics Server), virtual nodes, workload identity and self-managed nodes all require them. Section 12 compares basic and enhanced clusters.
- **Office hours:** Monday to Friday, 08:00 to 20:00 America/Sao_Paulo (UTC−3, no daylight saving), 60 of the 168 hours in a week.
- **Prices:** OCI public price list API, pay-as-you-go USD, fetched 2026-10-27. Negotiated discounts change the absolute numbers, not the ratios.
- **Cost model simplification:** cost is attributed to requested resources. This assumes nodes are well packed and the node autoscaler removes capacity. Real savings are lower by your packing inefficiency.
- **Lab:** kind v0.33 with Kubernetes 1.36.4 on containerd 2.3.4, VPA 1.8.1, KEDA 2.21.0 with the HTTP add-on 0.16.0, GoKubeDownscaler 1.4.2, metrics-server 0.9.0. Runtimes: Node.js 24.22.1 and 26.12.1, PHP 8.4 FPM, Go 1.27. OKE uses CRI-O instead of containerd, and the lab cannot test OCI node provisioning, so only pod-level behaviour was verified.

3. How the money flows on OKE

Three layers decide the bill. "Near zero" means working all three.

Layer	What changes	Tools	Saves money when
Replicas (horizontal)	Number of pods	HPA, KEDA	Fewer pods means fewer requests
Pod size (vertical)	Requests/limits per pod	In-place resize, VPA, startup boost, shrinker	Smaller requests
Capacity (nodes)	Number, shape and price of nodes	Cluster Autoscaler, Karpenter (KPO), flex/ burstable/preemptible/Arm shapes	Nodes are removed or replaced by cheaper ones

The first two layers only lower *requests*. The third layer is where requests become money. If no node goes away, shrinking pods saves nothing.

4. What OKE and Kubernetes already provide

Feature	Status on OKE (Oct 2026)	Role in "near zero"	Notes
In-place pod resize	GA on OKE 1.35.2 / 1.36.4; beta and on by default on 1.34.10	Shrink and grow running pods without restart	CPU and memory only; QoS class cannot change
Vertical Pod Autoscaler	You install it (Helm chart or upstream script). OKE guide	Measure and right-size requests	Not supported on virtual nodes. The OKE guide still uses VPA 0.8 and the deprecated `Auto` mode; use VPA 1.8.x
HPA	Native	Keep `minReplicas: 1`, scale on load	Use `AverageValue`, not `Utilization`, when requests change
HPA scale to zero	Beta in K8s 1.37 (blog)	True zero for non-prod	OKE has 1.37 only as a preview
Metrics Server	OKE add-on	Needed by HPA and VPA	
Cluster Autoscaler	OKE add-on or standalone (docs)	Removes nodes after requests drop; pools can go to 0	One shape per node pool; keep a system pool it does not manage
Karpenter Provider for OCI (KPO)	GA April 2026, v1.4.0 (docs, GA blog)	Consolidates under-used nodes, picks shapes, supports burstable and preemptible	Supports K8s 1.31 to 1.36
Flexible shapes	Managed and KPO nodes	Pick the OCPU:memory ratio that matches your requests	1 to 64 GB per OCPU on E4/E5
Burstable instances	KPO (`baselineOcpuUtilization`) and self-managed nodes	Pay for 12.5% or 50% of the OCPUs	Memory is billed in full; bursts last at most about 1 h (docs)
Preemptible capacity	Managed node pools and KPO	50% cheaper nodes	Reclaimed at any time, about 2 min notice; cannot be burstable (docs)
Ampere A1/A2 (Arm)	Managed and KPO nodes	Cheaper CPU	Needs multi-arch images
Virtual nodes	Enhanced clusters	Serverless burst capacity	Poor fit for tiny pods (section 9.4)

I found no field for the burstable baseline in the OKE managed node pool API: its `NodeShapeConfig` only exposes `ocpus` and `memoryInGBs`. Burstable workers therefore come through KPO or self-managed nodes.

Open-source tools around this:

Tool	Version checked	Use
KEDA	2.21.0	Cron and event triggers; can keep `minReplicaCount: 1`. Oracle shows KEDA plus the Cluster Autoscaler on OKE in this blog
KEDA HTTP add-on	0.16.0 (beta)	Holds requests while a service scales from zero. Fits non-prod only, since it still waits for the cold start
kube-startup-cpu-boost	0.21.3	Temporary CPU boost at start-up via in-place resize
Goldilocks	4.16.2	Dashboard over VPA recommendations
KRR	1.31.1	Request recommendations from Prometheus history, no VPA needed
OpenCost	1.121.3	Per-namespace and per-workload cost; supports OCI pricing
GoKubeDownscaler / kube-green	1.4.2 / 0.7.1	Office-hours scale-to-zero for non-prod

Tool	Version checked	Use
Elastic CPU Pod Autoscaler (DoorDash)	n/a	Fast, CPU-only in-place autoscaling

Commercial products also implement the "shrink the last replica" idea directly. One example is Kedify's PodResourceProfile, which resizes requests in place when a KEDA ScaledObject turns inactive. CAST AI, ScaleOps and others offer in-place rightsizing too. I did not evaluate them.

5. Vertical scaling on OKE

5.1 In-place pod resize: the mechanism

You change `spec.containers[*].resources` through the pod's `resize` subresource, and the kubelet applies the new values to the running container's cgroup (Kubernetes docs).

```
kubectl -n demo patch pod my-app-xxxx --subresource resize \
  -p '{"spec":{"containers":[{"name":"app","resources":{"requests":{"cpu":"25m","memory":"64Mi"}}}]}}'
```

Value	Linux cgroup v2 effect	Effect of lowering it
CPU request	`cpu.weight` (share under contention)	Less CPU only when the node is busy; scheduler sees more free room
CPU limit	`cpu.max` (hard quota)	Throttling above the new limit
Memory request	Scheduling and eviction ranking only (unless the `MemoryQoS` feature gate is enabled)	Scheduler sees more free room; pod moves up the eviction queue if usage > request
Memory limit	`memory.max`	OOM kill if usage exceeds it

Rules that matter for this pattern:

- **The QoS class cannot change.** A Guaranteed pod (requests = limits) can only be resized with requests still equal to limits. To lower requests while keeping limits, the pod must be Burstable from the start (`examples/01-app-deployment.yaml`).
- **`resizePolicy`** per resource is `NotRequired` (default, no restart) or `RestartContainer`.
- **Memory limit decreases are best effort.** If current usage is above the new limit, the kubelet holds the resize as "in progress" and does not apply it.
- **Use strategic merge patch (the kubectl default) or JSON patch.** `--type merge` replaces the whole containers list, and the API server rejects the request. **(lab)**
- **The Deployment template is not changed.** New pods (scale-out, rescheduling after a node drain) start at template size. GitOps tools see no drift.
- RBAC for automation needs `get` and `patch` on `pods/resize`. **(lab)**
- Kubernetes 1.36 adds beta in-place resize of pod-level resources (`spec.resources`), which helps multi-container pods (blog).

Lab results (Kubernetes 1.36.4):

Test	Result
Requests 500m/256Mi → 25m/64Mi, limits unchanged	Applied, `restartCount` 0. `cpu.weight` went from 59 to 6; `cpu.max` and `memory.max` unchanged
Lower requests to equal limits (Burstable → Guaranteed)	Rejected: "Pod QOS Class may not change as a result of resizing"

Test	Result
Memory request 64Mi on a container using ~200Mi	Applied, no restart
Memory limit 400Mi → 128Mi on a container using ~200Mi	Not applied; condition `PodResizeInProgress`; actual limit stayed 400Mi
Memory limit 400Mi → 300Mi (above usage)	Applied, no restart
Raise requests back to 500m/256Mi	Applied, no restart

One more lab observation: after lowering the memory request, the container's `oom_score_adj` stayed at the value computed from the original request. Do not count on a smaller request making the pod a more likely kernel OOM victim; the kubelet's eviction ranking does use the current request.

5.2 Vertical Pod Autoscaler

Install on OKE with the official Helm chart instead of the 0.8-era script in the OKE guide:

```
helm repo add autoscaler https://kubernetes.github.io/autoscaler
helm upgrade --install vpa autoscaler/vertical-pod-autoscaler --version 0.15.0 -n kube-system
# Metrics: enable the OKE "Kubernetes Metrics Server" cluster add-on first.
```

Update modes in VPA 1.8 (API reference):

Mode	Behaviour	Use it for
`Off`	Recommends only	Always start here (`02`)
`Initial`	Sets requests at pod creation only	Combining with another controller that resizes later
`Recreate`	Evicts pods to apply changes	Legacy; `Auto` is deprecated and means the same
`InPlaceOrRecreate`	Resizes in place, evicts only if that fails	Default choice (GA since VPA 1.6) (`03`)
`InPlace`	Resizes in place, never evicts	Alpha (VPA 1.7, `--feature-gates=InPlace=true`)

Recommended policy for this use case (from `examples/03-vpa-inplace.yaml`):

```
spec:
  updatePolicy:
    updateMode: "InPlaceOrRecreate"
    minReplicas: 1                    # allow updates on single-replica workloads
  resourcePolicy:
    containerPolicies:
      - containerName: app
        controlledValues: RequestsOnly # keep limits as written
        minAllowed: { cpu: 25m, memory: 128Mi }
        maxAllowed: { cpu: "2", memory: 2Gi }
```

`RequestsOnly` keeps the memory limit as a fixed safety ceiling. It also keeps the CPU limit, which the JVM and other runtimes read at start-up to size their thread pools.

What VPA is and is not good at. The recommender keeps an 8-day, exponentially decaying usage histogram (flags):

- CPU target is the 90th percentile of usage.
- Memory target is the 90th percentile of daily *peaks*.
- 15% safety margin on top of both.
- Floors of 25m CPU and 250 MiB memory per pod.

So VPA sizes a pod for its busy day. It does not shrink it for the night. Treat VPA as the right-sizing tool and use a separate mechanism (5.4) for idle periods.

Lab results:

- With the upstream "hamster" workload (requests 100m, burns about 500m), `InPlaceOrRecreate` raised the request to 587m within about two minutes on the same pod, with zero restarts.
- VPA also raised the memory request from 50Mi to 250Mi because of the 250 MiB default floor (the flag is named `-mb`, but the lab showed it applied as `250Mi`). For very small pods, lower `--pod-recommendation-min-memory-mb` on the recommender.

5.3 CPU startup boost: small steady state, fast start

A small CPU request slows down start-up when the node is busy, and a low CPU limit slows it down every time. JIT-compiled runtimes suffer most. A startup boost gives the pod more CPU until it is Ready, then shrinks it in place. That makes a tiny steady-state request compatible with short cold starts, and it also speeds up scale-out from 1 replica.

Option	Status	Example
VPA `startupBoost`	Alpha, VPA 1.7+, gate `CPUStartupBoost` on admission controller and updater	`04-vpa-startup-boost.yaml`
kube-startup-cpu-boost operator	v0.21.3, independent of VPA	`10-kube-startup-cpu-boost.yaml`

Lab results:

- VPA boost with factor 4: the CPU request went from 500m to 999m at creation. `RequestsOnly` caps the boost just below the 1-CPU limit to keep the QoS class. 30 s after Ready, VPA resized the pod in place to its recommendation (25m), with zero restarts.
- kube-startup-cpu-boost applied the boost at admission and reverted it 3 s later, when the pod became Ready.

Caveats (kube-startup-cpu-boost README):

- If the boost raises the CPU *limit*, a JVM sizes its pools for the boosted CPU count and keeps them after the revert. Keep boosts modest or set `-XX:ActiveProcessorCount`.
- A boosted request can trigger a node scale-up that is useless a minute later. Keep some headroom or slow down scale-down.

5.4 Shrinking the warm pod during idle periods

Two ways to do it:

a) Schedule-based (predictable office hours). `examples/05-idle-shrinker-cronjob.yaml` is a small, dependency-free implementation:

- At 20:00 Mon–Fri, a CronJob lowers the requests of opted-in pods to the values in their `idle-shrink.example.com/requests` annotation.
- At 07:30, a second CronJob restores the requests from the owning ReplicaSet template.
- Limits are never touched.
- Memory guard: the job never sets a memory request below current usage + 15%. If metrics are missing, it leaves memory alone.

Lab: shrink and restore both worked with zero restarts. The memory guard raised a 32Mi idle target to 230Mi for a container using 200Mi.

b) Event-based (unpredictable traffic). Shrink when the ScaledObject goes inactive and grow on activation. There is no mature open-source controller for this yet. Kedify PodResourceProfile does it commercially, and DoorDash's EPCAS covers the CPU-only case. Alternatively, keep the warm pod at a "small but responsive" size (for example 100m CPU) and let the HPA add full-size pods. Section 10 covers how to combine schedules with unpredictable traffic.

Do not combine (a) with VPA in `InPlaceOrRecreate`, `Recreate` or `InPlace` mode on the same workload, because VPA reverts the change. VPA `Off` or `Initial` is fine.

5.5 Interaction rules

Combination	Safe?	Why
HPA/KEDA on CPU `Utilization` + any request shrinking	No	Example: request 500m with a 70% target scales out at 350m. After shrinking to 25m, it scales out at 17.5m
HPA/KEDA on CPU `AverageValue`, request rate, or queue length + VPA/shrinker	Yes	Signal does not depend on requests (`06`, `07`)
VPA (updating) + HPA on the same CPU/memory metric	No	Upstream VPA and OKE both advise against it
VPA (updating) + idle shrinker on the same pods	No	Two writers fight
VPA startupBoost + kube-startup-cpu-boost on the same pods	No	Pick one
VPA (updating) + KEDA cron with `minReplicaCount: 1`	Yes	Different layers

6. CPU or memory: which reduction saves more?

6.1 Price per unit

Kubernetes `1` CPU = 1 vCPU. On x86 shapes, 1 OCPU = 2 vCPUs. On Ampere A1, 1 OCPU = 1 vCPU. On A2, 1 OCPU = 2 cores. Appendix C.4 shows step by step how these prices become the monthly figures below.

Shape	USD per vCPU-hour	USD per GB-hour	GB of RAM that cost as much as 1 vCPU
E4 Flex (AMD)	0.0125	0.0015	8.3
E5 Flex (AMD)	0.0150	0.0020	7.5
E6 Flex (AMD)	0.0150	0.0020	7.5
A1 Flex (Ampere)	0.0100	0.0015	6.7
A2 Flex (AmpereOne)	0.0070	0.0020	3.5

Typical services request 1 to 4 GB per vCPU, far below these break-even ratios. CPU is the bigger part of the bill unless a pod requests more than about 7 GB per vCPU on AMD shapes.

6.2 Where the money goes in one pod (E5 Flex, 24x7)

Requests	CPU USD/month	Memory USD/month	CPU share
1000m / 2Gi	10.95	3.14	78%
500m / 1Gi	5.47	1.57	78%
250m / 1Gi	2.74	1.57	64%
50m / 1Gi	0.55	1.57	26%

Once CPU is trimmed, memory becomes most of what is left. Memory is also the resource you can only reduce further by changing the application (6.4).

6.3 What happens when each one is too low

CPU too low:

- Below the request, nothing happens. Under contention a container gets CPU in proportion to its request, and spare CPU is shared.
- At the limit, the kernel throttles the container: latency goes up, and nothing is killed.
- Indirect risk: a liveness probe that times out under throttling restarts the container. Give probes generous timeouts and do not run heavy work inside them.

Memory too low. Three separate mechanisms can kill or move a pod (node-pressure eviction):

Mechanism	Trigger	Result	Controlled by
Container OOM kill	Container usage > its memory **limit**	Kernel kills the process; kubelet restarts the container (cold start)	Memory limit
Kubelet eviction	Node memory available falls below the eviction threshold	Pod deleted and recreated elsewhere (cold start). Pods whose usage exceeds their **request** go first, then by priority, then by how far over request they are	Memory request, PriorityClass
Node OOM	Node runs out before the kubelet reacts	Kernel kills by `oom_score_adj` : BestEffort 1000, Burstable 2–999 (smaller request = higher score), Guaranteed −997	QoS class, request

So the concern that pods get killed when memory is too low is correct, and it splits in two:

- **Limits decide OOM kills.** Keep the memory limit at peak + headroom and never lower it in place below current usage. The kubelet refuses to apply such a change **(lab)**, but a usage spike right after the check is still possible.
- **Requests decide eviction order and how tightly the scheduler packs.** A memory request below real usage frees no RAM. It only lets the scheduler put more pods on the node and makes this pod an early eviction victim. That saving is fake unless usage actually went down.

6.4 Idle behaviour is different

- **CPU:** an idle service uses a few millicores. Lowering its request from 500m to 25–50m matches reality, so the saving is real and low-risk.
- **Memory:** an idle service keeps its long-lived working set (loaded code, caches, connection pools). Memory used during the last burst may or may not go back to the operating system, depending on the runtime. Lab measurements after a burst, with no further traffic (section 7):

Runtime	Right after the burst	Later, with no traffic
PHP-FPM, `pm = ondemand`	172 MiB	10 MiB after 25 s
PHP-FPM, `pm = static`	172 MiB	172 MiB, does not drop
Node.js 24	about 200 MiB	about 60 MiB after about 2 min
Go 1.27	157 MiB	10 MiB after about 4.5 min

- Kubernetes cannot make a process give memory back; runtime settings decide it. So measure idle memory a few minutes after traffic stops, and set the off-hours memory request from that number.
- Other runtimes the platform may host: the JVM sizes its heap from `-Xmx` or `MaxRAMPercentage` at start-up, and G1 returns unused heap when idle with `-XX:G1PeriodicGCInterval` (ZGC and Shenandoah do it by default). .NET Server GC keeps large heaps; DATAS (default since .NET 9) or `GCConserveMemory` shrink them.
- In-place memory resize is therefore mostly useful for *requests* (scheduling), not for the process's actual footprint.

6.5 Scenario comparison (30 services, E5 Flex)

Business hours are Mon–Fri 08:00–20:00 (60 h/week) with 3 replicas × (500m, 1Gi). The rest of the week (108 h) varies:

Off-hours strategy	USD/month	Saving vs always-on
A. No change, 3 replicas	634	0%
B. 1 replica, full size	362	43%
C. 1 replica, CPU request 50m, memory 1Gi	267	**58%**
D. 1 replica, CPU 500m, memory request 512Mi	347	45%
E. 1 replica, CPU 50m + memory 768Mi	260	59%
F. 0 replicas (true scale-to-zero)	226	64%

Halving memory (D) adds 2 points over B. Cutting CPU (C) adds 15. C reaches about 90% of F's saving without any cold start.

6.6 Verdict

1. **CPU saves more and is safer to cut.** Shrink idle CPU requests to 25–50m. Keep a CPU limit for JVM services (it fixes the processor count the JVM sees), or set `-XX:ActiveProcessorCount`. For other runtimes the limit is optional.
2. **Memory:** set the request to the measured idle working set + 10–15% and the limit to peak + headroom. Get further reductions from runtime tuning, not from Kubernetes.
3. **After CPU is trimmed, memory dominates the remaining cost.** Pick node shapes with more GB per OCPU (flex shapes allow up to 64 GB/OCPU) so you do not pay for idle CPU on the nodes themselves.

7. Runtime guidance: Node.js, PHP and Go

The platform will mostly host Node.js, with PHP and Go. Each runtime reads the container limits in its own way, and that decides what you can safely change in place. None of the three looks at CPU or memory *requests*, which is why shrinking requests is safe for all of them.

	Node.js 24	PHP-FPM 8.4	Go 1.27
Reads the CPU limit	Yes: `os.availableParallelism()`, follows in-place changes **(lab)**	No; concurrency is `pm.max_children`	Yes: GOMAXPROCS = CPU limit (minimum 2), follows in-place changes within about 1 s **(lab)**
Reads the memory limit	Yes, once: V8 heap is about 50% of the limit at start-up; not recalculated after a resize **(lab)**	No	No; set `GOMEMLIMIT` yourself
Idle memory after a burst **(lab)**	about 200 → 60 MiB after about 2 min	`ondemand`: 172 → 10 MiB in 25 s; `static`: stays at 172 MiB	157 → 10 MiB after about 4.5 min
Wake from zero, minimal server, image already on the node **(lab)**	0.8–1.9 s	1.1–1.4 s (with nginx)	1.0–2.1 s
Safe to change in place	CPU requests and limits, memory requests	CPU requests and limits, memory requests	CPU requests and limits, memory requests
Change only with a rollout	Memory limit (heap size follows it)	Memory limit, together with `pm.max_children`	Memory limit, together with `GOMEMLIMIT`
Example	`11a`	`11b`	`11c`

7.1 Node.js

- **CPU:** JavaScript runs on one thread per process, so a process uses about one core, plus some garbage-collector and libuv threads. Set the CPU limit to 1 (2 if you use `worker_threads`) and scale out with replicas. Avoid running a PM2 or `cluster` fleet inside one pod: each worker has its own heap, and the pod's idle footprint grows with the worker count.
- **Heap size:** V8 takes its heap limit from the container memory limit when the process starts.
  - Lab: a 512Mi limit gave a 259 MiB heap on Node 24.22 and 268 MiB on Node 26.12. A pod started with 1Gi got 560 MiB.
  - `NODE_OPTIONS=--max-old-space-size-percentage=75` raised it to 387 MiB at 512Mi. 70–75% leaves room for buffers, native modules and the libuv pool.
  - After an in-place resize from 512Mi to 1Gi, the heap limit stayed at 259 MiB (and 387 MiB with the flag), while `process.constrainedMemory()` reported the new 1 GiB. An open Node.js issue asks for dynamic recalculation (nodejs/node#65235).
  - Consequence: shrink memory *requests* in place freely; change memory *limits* only through a rollout.
- **Idle memory:** after a burst, V8 kept about 120 MiB of garbage for about 100 s, then RSS fell from about 200 MiB to 60 MiB without any traffic. A forced `global.gc()` emptied the heap but did not lower RSS right away. Measure idle RSS at least 3–5 minutes after traffic stops. The shrinker in example 05 uses current usage as its memory floor, so it adapts.
- **Start-up time:** set `NODE_COMPILE_CACHE` (Node 22.1+) so V8 reuses compiled code. A cache written at run time only helps container restarts; generate it at image build time so new pods benefit too (portable cache since Node 24.12, read-only cache since 26.8; docs). Start-up is mostly single-threaded module loading, so a CPU boost above 1 CPU helps little; a boost up to 1 CPU helps when the steady-state limit is lower.
- **Probes:** a throttled event loop answers late. Give liveness probes generous timeouts (example 11a uses 5 s and 6 failures).
- **Versions:** Node.js 24 is the active LTS (24.22.1), 22 is in maintenance (22.23.4), 26 is current (26.12.1) and becomes LTS at the end of October 2026.

7.2 PHP

PHP runs in one of two models, and they behave very differently at idle:

Model	Examples	Process model	Idle memory
Classic, per request	nginx + PHP-FPM, FrankenPHP classic mode	The app boots on every request; workers are processes (FPM) or threads	Can drop to almost nothing with `pm = ondemand`
Long-running	FrankenPHP worker mode, Laravel Octane (Swoole, RoadRunner, FrankenPHP)	The app boots once and stays in memory	Stays, like Node.js

PHP-FPM:

- **Use `pm = ondemand`.** Workers start when requests arrive and exit after `pm.process_idle_timeout` (default 10 s) without work (PHP-FPM configuration). Lab, 8 concurrent clients for 20 s, each request building about 24 MiB of small strings:
  - `pm = static`, 8 workers: 172 MiB right after the load, still 172 MiB at idle.
  - `pm = ondemand`, up to 8 workers: 172 MiB right after the load, 0 workers and 10 MiB 25 s later.
  - With a single large buffer per request, both modes dropped back after each request, because PHP frees large blocks at request end. The gain depends on how the app allocates; measure yours.
- **Cost of `ondemand`:** the first request after an idle period forks a worker, which takes milliseconds. OPcache lives in shared memory owned by the master, so compiled code survives.
- **Memory limit** ≥ `pm.max_children` × peak worker RSS + master and OPcache + headroom. `pm.max_children` is read at start-up; an in-place resize does not change it, so change both together through a rollout.

- **CPU:** one request occupies one worker process. Set the CPU limit to the concurrency you want per pod (1–2 CPUs) and scale out with replicas on request rate.

Long-running PHP:

- FrankenPHP is a Go program that embeds PHP. Its docs recommend setting `GOMEMLIMIT` in containers, setting `num_threads` / `max_threads` explicitly (default threads are 2× the CPU cores), keeping `max_threads × memory_limit` below the container memory, and using glibc-based images rather than Alpine/musl for speed (FrankenPHP performance). The Go notes in 7.3 apply.
- Octane and FrankenPHP worker mode trade a faster request path for a resident app: size and shrink them like Node.js services.

Queues: Laravel's Redis queue keeps pending jobs in a Redis list (`queues:default`), which KEDA's `redis` scaler can watch to run workers from zero (example 14, section 11).

Versions: PHP 8.5.12, 8.4.27 and 8.3.36 are current (October 2026).

7.3 Go

- **GOMAXPROCS follows the CPU limit** since Go 1.25 (release notes, blog):
  - The default is the CPU limit rounded up, but never below 2 (unless the machine has fewer CPUs). CPU requests are ignored.
  - The runtime re-checks at most once per second. Lab: an in-place change from limit 1 to limit 3 gave GOMAXPROCS=3 about 1 s later, without a restart. Limit 2 → 1 stayed at 2, the documented floor.
  - It only applies when `go.mod` declares `go 1.25` or later and the `GOMAXPROCS` environment variable is not set. Older `go.mod` files keep the old default, the node's core count. On a 64-core node that means 64 threads competing for a 1-CPU quota, which causes throttling.
  - Consequence: set a CPU limit on every Go container (or set `GOMAXPROCS`), then shrink requests freely.
- **GOMEMLIMIT is not automatic** in Go 1.25 to 1.27. A value from the downward API is fixed when the container starts: after an in-place resize from 768Mi to 1Gi the runtime still reported 768 MiB **(lab)**. Set it to about 90% of the memory limit explicitly, or use the `automemlimit` package.
- **Idle memory:** after a burst, Go's forced GC (every 2 minutes) emptied the heap, and the background scavenger returned the pages to the OS about 4.5 minutes after the burst (157 → 10 MiB).
- **Start-up:** static binaries in `scratch` or distroless images are small (5.9 MB in the lab) and start in milliseconds, so Go services are the best candidates for true scale-to-zero.
- **Versions:** Go 1.27.3 and 1.26.10 are the supported releases.

7.4 Cold start: what the lab numbers mean

The 0.8–2.1 s wake-from-zero times above were measured through the KEDA HTTP interceptor, for minimal servers, on a node that already had the image. Most of that time is Kubernetes itself: scheduling, creating the pod sandbox, starting the container, and the first readiness probe (1 s period). A real cold start adds:

Step	Typical size	How to cut it
Node provisioning, if no node has room	Minutes on OKE	Keep spare room with placeholder pods (section 10.2); OKE images instead of platform images
Image pull, if the node lacks the image	Seconds to tens of seconds	Small images (distroless, Alpine), same base image across services, pre-pull with a DaemonSet
Application start-up	Framework boot, connections, cache warm-up	`NODE_COMPILE_CACHE`, OPcache preload, lazy initialisation, startup boost
Readiness	Probe period × failures	Short `startupProbe` period (1 s)

Measure each app before deciding, from the pod's own timestamps:

```
kubectl -n <ns> get pod <pod> -o json \
  | jq -r '"created \(.metadata.creationTimestamp)", (.status.conditions[] | "\(.type) \
(.lastTransitionTime)")'
```

If an app is Ready within 1–2 s of `PodScheduled` and its image is already on the nodes, scaling it from zero costs the first user about 2–4 s. Whether that is acceptable is a product decision; the measurement tells you which services it applies to.

8. Horizontal layer: never zero, rarely more than needed

`examples/06-keda-cron-min-one.yaml`:

```
spec:
  minReplicaCount: 1              # one warm pod, no cold start for the first user
  maxReplicaCount: 10
  triggers:
    - type: cron                  # pre-warm before users arrive
      metadata: { timezone: America/Sao_Paulo, start: "15 7 * * 1-5", end: "0 20 * * 1-5",
desiredReplicas: "3" }
    - type: cpu
      metricType: AverageValue    # not Utilization: immune to request changes
      metadata: { value: "400m" }
```

Lab: KEDA created an HPA with the cron external metric and `cpu: AverageValue 400m`, as expected.

Practical points:

- **Pre-warm, then restore.** Run the business-hours restore (5.4) and the cron scale-up 15–30 minutes before traffic arrives, so the morning ramp never waits for a cold start.
- **Request-rate triggers** (Prometheus scaler, or the KEDA HTTP add-on's metrics) follow user load more closely than CPU.
- **Fast scale-out from 1** also needs node capacity. Adding an OKE node takes minutes (OKE images cut provisioning time by more than half compared with platform images). For latency-sensitive services, keep a few low-priority placeholder pods: real pods preempt them instantly, and the autoscaler adds a node for the evicted placeholders in the background (section 10.2, example `15`).
- **Non-production can go to real zero.** Nobody waits for a cold start at night. Use the Oracle-documented pattern of KEDA cron (+ HTTP add-on for wake-on-request) plus a node pool with min 0 (Oracle blog), or GoKubeDownscaler. Section 11 lists which workloads qualify and what still costs money at zero.

9. Node layer: turning smaller requests into a smaller bill

9.1 Cluster Autoscaler (OKE add-on)

Configuration: `examples/09-oke-cluster-autoscaler-addon.json`.

```
oci ce cluster install-addon --cluster-id <cluster-ocid> --addon-name ClusterAutoscaler \
  --from-json file://examples/09-oke-cluster-autoscaler-addon.json
```

Settings that matter here (add-on arguments):

Key	Default	Why it matters
`scaleDownUtilizationThreshold`	0.5	Node "utilization" is the sum of **requests** / capacity. Shrinking requests is what makes nodes removable. 0.6 removes nodes a bit more eagerly

Key	Default	Why it matters
`scaleDownUnneededTime`	10m	How long a node must stay under the threshold
`skipNodesWithLocalStorage`	true	Pods with `emptyDir` block scale-down. Set to false if emptyDir content is disposable
`nodes`		`min:max:pool-ocid`. Prod app pools min ≥ 1, non-prod pools min 0

OKE best practice: keep one node pool that the autoscaler does not manage for system pods and the autoscaler itself (best practices). Known issue: when a pool scales up from zero, the autoscaler overestimates the CPU of Arm flex shapes by 2x, which costs one extra scaling cycle (kubernetes/autoscaler#9337).

9.2 Karpenter Provider for OCI

`examples/08-karpenter-oci-burstable-and-preemptible.yaml` (validated against the KPO v1.4.0 CRDs) defines:

- a **low-cpu** pool on burstable E5/E4 Flex nodes, with consolidation `WhenEmptyOrUnderutilized`;
- a disruption budget that blocks consolidation from 07:00 to 19:00 Mon–Fri (São Paulo time). Karpenter budget schedules have no time zone support, so they are written in UTC;
- a **preemptible** pool (Karpenter `spot`, which KPO maps to OCI preemptible), which requires the `oci.oraclecloud.com/oke-is-preemptible` taint.

Compared with the Cluster Autoscaler, Karpenter can replace a half-empty node with a smaller one and mix shapes in one policy. That matters when off-hours requests shrink unevenly. Do not let both autoscalers manage the same pods (migration guide).

9.3 Consolidation means cold starts

Removing a node means evicting its pods, and every evicted pod pays a cold start. Mitigations:

- Run 2 replicas with a PodDisruptionBudget (`maxUnavailable: 1`) during business hours. The second replica is cheap after CPU trimming.
- Allow consolidation only in the evening window (Karpenter budgets), right after the shrink, while traffic is low.
- Keep cold starts short with a startup boost (5.3).
- For a single-replica service that must never move, annotate it with `cluster-autoscaler.kubernetes.io/safe-to-evict: "false"` or `karpenter.sh/do-not-disrupt: "true"`. That node then stays, so use it sparingly.

9.4 Node price levers for the warm pod (50m / 1Gi, 24x7)

Placement	USD/month
E5 Flex, on-demand	2.12
E5 Flex burstable, 50% baseline	1.84
E5 Flex burstable, 12.5% baseline	1.64
A1 Flex (Arm), on-demand	1.54
Virtual node (Pod.Standard.E4.Flex)	20.44 + US$0.015/h per virtual node

- **Burstable nodes** are CPU overcommit priced in by OCI:
  - Kubernetes sees all vCPUs, and you pay for the baseline.
  - If the node's *average* CPU stays below the baseline, every pod gets what it asks for. If it stays above, OCI caps the whole node and every pod slows down.
  - They fit non-prod and services that idle most of the time. Example 08 also shows the safer `kubeReserved` variant, which hides the non-baseline CPU from the scheduler.

    - For a CPU-trimmed pod the gain is small (memory dominates). The gain is large for pools where requests stay high but usage is low.
- **Virtual nodes** do not fit tiny pods. Requests must equal limits, sub-core sizes are not supported, allocation rounds up to whole OCPUs and whole GB plus 0.25 GB overhead, and VPA does not work there (resource allocation, comparison). They are useful for stateless burst capacity on top of managed nodes (Oracle mixed-cluster blog).
- **Preemptible (−50%)** suits the 2nd to Nth replicas of stateless services, batch and CI. It never suits the only warm replica.
- **Arm (A1/A2)** has the cheapest CPU if your images are multi-arch.

9.5 Pod density: when IP addresses, not CPU or memory, limit packing

Once pods are tiny, the number of pods a node may run can become the limit before CPU or memory:

- **VCN-native pod networking:** each pod gets a VCN IP from the node's secondary VNICs. Maximum pods per node = MIN((VNICs − 1) × 31, 256) in the standard configuration (OKE docs). E5 Flex VMs get 2 VNICs at 1 OCPU and 1 VNIC per OCPU from 2 OCPUs (burstable shapes table), so 1- and 2-OCPU nodes hold 31 pods and a 4-OCPU node 93. Since 2026 you can attach several secondary VNIC profiles and reach 256 pods per node (release note).
- **flannel:** pods use an overlay network, and the pod count is not tied to the shape (OKE docs); the kubelet default of 110 applies.

Cost per tiny pod (25m CPU / 128Mi requests, 4 slots per node taken by DaemonSets, E5 Flex, 24x7):

Node	USD/ month	Pods that fit by CPU/ memory	VCN-native max	flannel max	USD per pod, VCN-native	USD per pod, flannel
1 OCPU / 8 GB	33.58	53	31	110	1.24	0.69
2 OCPU / 16 GB	67.16	107	31	110	2.49	0.65
4 OCPU / 32 GB	134.32	214	93	110	1.51	1.27
8 OCPU / 64 GB	268.64	429	217	110	1.26	2.53

With VCN-native networking, avoid 2-OCPU nodes for pools of tiny pods. With flannel, avoid nodes so large that 110 pods leave CPU and memory idle (or raise `max-pods`). The CNI is chosen when the cluster is created and cannot be changed later (section 12.4).

10. Unpredictable traffic

Schedule-based shrinking (section 5.4) is a bet that traffic follows office hours. The team expects that, but it is not confirmed. The setup below keeps the cost of losing that bet small, and section 10.3 shows how to check it with data.

10.1 Make a wrong guess cheap

- **Shrink requests, never limits.** The limit is what a container may use; the request only matters when the node is full. **(lab)** A pod whose CPU request was shrunk in place to 25m, with a 1-CPU limit, still used 0.91–0.99 CPU on a node with free CPU. A night spike on a shrunk pod is served at full speed as long as the node has spare CPU.

- **The real risk is contention.** At night the pods are packed tighter, which is the point of the saving. If several shrunk pods spike at the same time on a full node, each gets CPU in proportion to its now small request. Three things cover that:
  - Horizontal scaling on absolute signals (CPU `AverageValue`, request rate, queue length; examples `06`, `07`). New replicas start at the full template size, because the shrinker only patches running pods.
  - Spare node room (10.2), so those new replicas start in seconds instead of waiting minutes for a node.
  - A startup boost (5.3), so the new replicas get through start-up quickly.
- **Memory stays safe.** The shrinker never sets a memory request below current usage + 15%, and limits do not change, so a spike does not add OOM-kill risk.
- **Shrink less if unsure.** Use the measured off-hours p95 rather than the minimum as the idle request. The saving drops a little and contention risk drops a lot.

10.2 Spare capacity with placeholder pods

`examples/15-capacity-placeholder.yaml` runs `pause` containers with a negative priority (-5) that reserve room on the nodes. When a real pod does not fit, the scheduler evicts a placeholder and starts the real pod at once. The evicted placeholder goes Pending, and the autoscaler adds a node for it in the background.

- **(lab)** With a placeholder holding 8.25 CPU and only 0.5 CPU left free, a pod requesting 2 CPU was Ready 1 s after creation, by preempting the placeholder.
- The priority (-5) must stay above the Cluster Autoscaler add-on's `expendablePodsPriorityCutoff` (default -10). Otherwise a Pending placeholder does not trigger a scale-up.
- You pay for the reserved room all the time. Size it to one full-size replica of the largest service per node pool. If night spikes turn out to be rare, scale the placeholders to 0 at night with the same tools as non-production (section 11).

10.3 Check the office-hours assumption

With Prometheus collecting cAdvisor metrics (for example kube-prometheus-stack, or the Prometheus that OpenCost needs), this query returns the share of CPU used outside office hours over the last 7 days. Prometheus works in UTC; São Paulo 08:00–20:00 is 11:00–23:00 UTC. Validated on Prometheus 3.15:

```
sum by (namespace) (sum_over_time((
    sum by (namespace) (rate(container_cpu_usage_seconds_total{container!=""}[5m]))
    and on() (hour() < 11 or hour() >= 23 or day_of_week() == 0 or day_of_week() == 6)
  )[7d:5m]))
/
sum by (namespace) (sum_over_time(
    sum by (namespace) (rate(container_cpu_usage_seconds_total{container!=""}[5m]))
  [7d:5m]))
```

Off-hours are 108 of the 168 hours in a week (64%). If CPU use were flat, the query would return about 0.64. The lower the value, the more traffic concentrates in office hours. Plot the inner expression (without `sum_over_time`) in Grafana to see the daily shape and the size of night spikes. If you have ingress metrics, the same query on request counts is an even better signal.

10.4 What to do with the result

What the data shows	Vertical	Horizontal
Clear office-hours pattern (off-hours share below about 0.25; a starting point, adjust to your data)	Schedule-based shrink (`05`)	KEDA cron pre-warm + absolute CPU (`06`)
Mostly quiet, with rare unpredictable bursts	Schedule-based shrink + placeholders (`15`)	KEDA or HPA on request rate; startup boost
Flat or random (share near 0.64)	VPA `InPlaceOrRecreate` right-sizing only (`03`); no schedule	HPA/KEDA on request rate or `AverageValue`, `minReplicas` 1–2

What the data shows	Vertical	Horizontal
Event-driven (queues, jobs)	Not needed	KEDA queue scaler with `minReplicaCount: 0` (14)

Different services can land in different rows. Run the query per namespace or per workload.

11. What can go to zero

11.1 What "zero" means

Scale-to-zero sets a workload's replica count to 0, so no pods run. That alone saves nothing on OKE, because you pay for nodes (section 3). The saving comes in the second step: when a node pool only ran those pods, the Cluster Autoscaler (pool minimum 0) or Karpenter removes the empty nodes, and their compute cost stops. Zero pods, then zero nodes.

The team ruled out zero for user-facing production services because of cold starts. Several other workloads have no user waiting, and those can go to zero today.

11.2 Candidates

Workload	Why zero is acceptable	How	Example
Non-production environments (dev, test, QA, staging, preview) outside working hours	Nobody uses them at night or at weekends	GoKubeDownscaler: one annotation per namespace	13
Non-production apps that someone may open at night	One user waits for one cold start	KEDA cron + HTTP add-on (wake on request)	12
Rarely used internal tools and admin UIs, any environment	A user can wait a few seconds once	KEDA HTTP add-on	12
Queue workers and consumers, every environment including production	Their "user" is a queue; a few seconds of delay is fine	KEDA queue scaler, `minReplicaCount: 0`	14
Scheduled jobs	They run and exit	Kubernetes CronJob (already zero between runs)	
User-facing production services	Not acceptable for now (team requirement)	Keep 1 warm, tiny pod (sections 5–10)	05, 06

Lab results:

- **Namespace downscaler (GoKubeDownscaler 1.4.2):** with `downscaler/uptime: "Mon-Fri 07:30-20:00 America/Sao_Paulo"` on the namespace, a 3-replica Deployment went to 0 outside the window and was restored to 3 about 30 s after uptime began; a workload marked `downscaler/exclude: "true"` kept running. Two defaults to know: new workloads are skipped for a 15-minute grace period, and the Helm chart only handles Deployments unless you add `statefulsets` to `includedResources`.
- **Queue worker (KEDA `redis` scaler):** the first worker was Ready 2–10 s after the first job arrived (KEDA polls every 10 s in the example), the worker scaled to 5 replicas for 100 jobs, and it went back to 0 about 2–3 minutes after the queue drained (`cooldownPeriod: 120`).
- **Wake on request (KEDA HTTP add-on 0.16, beta):** the first request to an app at zero replicas took 1.0–1.7 s for nginx, and 0.8–2.1 s for minimal Node.js, PHP-FPM and Go servers, on a node that already had the image.

Notes:

- KEDA 2.21 has no OCI-specific scaler. Use the scaler of the broker you run (Redis, RabbitMQ, Kafka, PostgreSQL, MySQL and others), or the `prometheus` / `metrics-api` scalers. OCI Streaming exposes a Kafka-compatible endpoint that KEDA's Kafka scaler may work with; I did not test it.

- KEDA, the HTTP add-on's interceptor and the autoscaler must run on a node pool that never scales to zero, as in Oracle's KEDA + Cluster Autoscaler guide.
- With the HTTP add-on, traffic must go through the interceptor proxy. Point the Ingress or Gateway of those apps at it.

11.3 What keeps costing money at zero

Item	USD/month
Enhanced cluster control plane (a basic cluster has no fee)	73.00
System node pool, 2 × E5 Flex 1 OCPU / 8 GB with 50 GB boot volumes	71.41
System node pool with a single node (non-production only, no redundancy)	35.70
Flexible load balancer, each one beyond the free hours	8.25
Network load balancer	0.00
Persistent volume, 50 GB balanced performance, each	2.12
Application node pool at 0 nodes	0.00

- The price list gives the first 744 load-balancer hours each month free, which is one load balancer running all month. The OCI Network Load Balancer is free (Oracle); OKE creates one for a `Service` of type `LoadBalancer` with the annotation `oci.oraclecloud.com/load-balancer-type: "nlb"` (OKE docs). It works at layer 4 only, so TLS and HTTP routing stay in your in-cluster ingress or gateway.
- Each separate enhanced cluster costs US$73/month even when empty. Run non-production environments as namespaces in one shared cluster, or as basic clusters (section 12.1).
- Persistent volumes and their data keep billing while the pods are at zero.

11.4 Getting the per-environment figures

- **On the OCI side:** give each environment its own node pool and apply a cost-tracking defined tag to the node pool's nodes (node tags are applied to the compute instances when they are created; existing nodes keep their old tags until they are replaced) (OKE node tags). OCI Cost Analysis can then group compute cost by that tag. A compartment per environment works too.
- **On the Kubernetes side:** OpenCost supports OCI pricing and splits node costs per namespace, workload and label, including environments that share nodes (OpenCost on OCI).
- Once you have the split: if non-production environments run all week today, taking them to zero outside office hours saves close to two thirds of their compute cost (the share of the week outside office hours). The fixed items in 11.3 remain.

12. Choosing the OKE cluster variant

The team has not decided the cluster setup yet. These are the choices that affect this strategy, and which of them cannot be changed later.

12.1 Basic or enhanced cluster

	Basic	Enhanced
Control plane price	Free	US$0.10/h, at most US$74.40/month
Availability commitment	Service level objective (SLO)	Financially backed SLA on the API server
OKE-managed add-ons (Cluster Autoscaler, Metrics Server, CoreDNS versions)	No: you install and upgrade them yourself	Yes

	Basic	Enhanced
Workload identity (IAM per Kubernetes service account)	No: instance principals, which grant more than needed	Yes
Virtual nodes, self-managed nodes	No	Yes
Node cycling on node pool updates and Kubernetes upgrades	No	Yes
Worker nodes per cluster	Lower limit	5,000; up to 20,000 with Oracle approval
Karpenter Provider for OCI	Not with the documented setup, which uses workload identity	Yes
Changing later	Can be upgraded to enhanced if its API endpoint is VCN-native; one-way	Cannot be downgraded

Sources: comparison, enhanced clusters, upgrading.

What it means here:

- Most of this report also works on a basic cluster: in-place resize, VPA, KEDA, GoKubeDownscaler, the shrinker, and the Cluster Autoscaler as a standalone deployment with instance principals. What you lose is the managed add-ons, workload identity, Karpenter Provider for OCI and virtual nodes.
- For production, the US$73/month is small next to node costs, and it buys the SLA, workload identity and node cycling.
- For non-production, prefer one shared enhanced cluster with a namespace per environment over many clusters. If you do need many short-lived clusters, basic clusters avoid US$73/month each. Create them with a VCN-native API endpoint so they can be upgraded later.

12.2 Node types

	Managed nodes	Virtual nodes	Self-managed nodes
What you pay for	Compute instances (OCPU + GB)	Each pod, rounded up to whole OCPUs and whole GB, plus US$0.015/h per virtual node	Compute instances
Requests below limits	Yes	No: requests must equal limits	Yes
In-place resize and VPA	Yes	VPA not supported; each pod has a fixed allocation	Yes
DaemonSets	Yes	No	Yes
Burstable / preemptible	Preemptible in node pools; burstable through Karpenter	Not applicable	Both (you create the instances)
Node autoscaling	Cluster Autoscaler or Karpenter	Not needed	Karpenter or your own tooling
Cluster type	Basic or enhanced	Enhanced	Enhanced
Fit for this strategy	Default choice	Burst capacity for stateless pods of 1 OCPU or more	Special shapes or images; more operations work

Source: virtual vs managed nodes. Virtual-node costs for small pods are in section 9.4.

12.3 Node autoscaler

	Cluster Autoscaler, OKE add-on	Cluster Autoscaler, standalone	Karpenter Provider for OCI
Cluster type	Enhanced	Basic or enhanced	Enhanced
Shapes	One per node pool	One per node pool	Many, chosen per pod from a policy

	Cluster Autoscaler, OKE add-on	Cluster Autoscaler, standalone	Karpenter Provider for OCI
Pools at zero nodes	Yes (pool minimum 0)	Yes	Yes (no fixed pools)
Removes under-used nodes	When requests fall below `scaleDownUtilizationThreshold`	Same	Consolidation: deletes or replaces with smaller nodes
Burstable / preemptible	Preemptible node pools	Same	Both
Status	Managed by Oracle	Upstream project	GA since April 2026 (v1.4.0)
When to use	Start here	On basic clusters	When you need several shapes, burstable nodes, or consolidation windows

Do not let two autoscalers manage the same pods (section 9.2).

12.4 Pod networking (CNI)

	flannel	VCN-native pod networking
Pod IP addresses	Overlay network, visible only inside the cluster	Real VCN IPs, reachable from peered VCNs and on-premises
Pods per node	Not tied to the shape (kubelet default 110)	MIN((VNICs − 1) × 31, 256); 31 on 1–2 OCPU E5 nodes
VCN IPs used	None for pods	One per pod, pre-allocated per node
Network rules per pod subnet (NSGs), VCN flow logs, Zero Trust Packet Routing	No	Yes
Virtual nodes	Not supported	Required
Default when creating a cluster	API	Console
Changing later	Not possible: the CNI is fixed at cluster creation (docs)	Not possible

For this strategy, pod density matters (section 9.5): with tiny pods, flannel packs up to 110 per node on any shape, while VCN-native needs larger nodes or extra VNIC profiles. Choose VCN-native if you need pods addressable from the VCN or on-premises, NSG-based pod isolation, or virtual nodes.

12.5 Kubernetes version

Version on OKE	In-place resize	Notes
1.34.10	Beta, enabled by default	Oldest supported; drops out about 30 days after the next version arrives
1.35.2	GA	Requires cgroup v2 (OKE OL8 images)
1.36.4	GA; pod-level resource resize in beta	Newest GA version on OKE; Karpenter Provider for OCI v1.3.0+
1.37.0	GA	Preview only on OKE; HPA scale-to-zero becomes beta

OKE supports three versions at a time; 1.33 lost support on 10 August 2026. Start new clusters on 1.36 and plan an upgrade every few months.

12.6 If you get to choose

- **Production:** enhanced cluster; Kubernetes 1.36; managed nodes on E5 or E6 Flex (A1/A2 Arm if your images are multi-arch); flannel unless you need VCN-native, and then no 2-OCPU nodes for tiny pods; a small system node pool that the autoscaler does not manage; the Cluster Autoscaler add-on first, Karpenter later for burstable nodes and consolidation windows.

- **Non-production:** one shared cluster with a namespace per environment; an application node pool with minimum 0 on preemptible capacity; GoKubeDownscaler on the namespaces; KEDA for workers and wake-on-request apps.

13. Suggested rollout

Phase	Actions	Exit criteria
0. Measure (1–2 weeks)	Every container has requests and limits. Install Metrics Server (add-on), VPA in `Off` mode (`02`), Prometheus, and OpenCost or Goldilocks. Tag node pools per environment (11.4). Measure each app's cold start (7.4). Run the office-hours query (10.3)	Idle and peak CPU/memory known per service; cost per environment visible; office-hours share known
1. Right-size	Apply VPA recommendations, through GitOps or `InPlaceOrRecreate` + `RequestsOnly` (`03`). Apply the runtime settings of section 7 (`11a`, `11b`, `11c`). Switch HPAs to `AverageValue` or request rate	Requests within ~20% of p90 usage; no OOM kills or throttling alerts
2. Zero where nobody waits	Non-production namespaces to zero outside office hours (`13`); queue workers to zero (`14`); internal tools on wake-on-request (`12`)	Non-production node pools at 0 nodes at night; no production risk
3. Warm but tiny in production	KEDA `minReplicaCount: 1` + cron pre-warm (`06`); idle shrinker (`05`) or VPA, not both; startup boost (`04` or `10`); placeholders if bursts are unpredictable (`15`). Pick the row of 10.4 per service	Night-time CPU requests down ≥ 80%; first-request latency unchanged
4. Capacity	Tune the Cluster Autoscaler add-on (`09`) or adopt Karpenter with consolidation windows (`08`); burstable pool for low-CPU services; preemptible for extra replicas; node sizes that suit the pod density limit (9.5)	Node count follows requests within ~30 min of the shrink
5. Cold start	Image size, compile cache and preload (7.4)	If a service's cold start is short enough, consider true zero for it

Phase 2 can start in parallel with phase 1: it needs no tuning of production services.

14. Risks and caveats

Risk	Mitigation
Memory request below real usage → early eviction under node pressure	Request ≥ idle usage + 10–15% (the shrinker enforces it); PriorityClass for critical services
Memory limit lowered below usage → OOM kill	Never lower limits in place; VPA `RequestsOnly`
Node.js heap sized from the start-up memory limit	Change memory limits only through a rollout; set `--max-old-space-size-percentage`
PHP-FPM `pm.max_children` × worker size above the memory limit → OOM kill under load	Size the limit from measured worker RSS; change both together
Go service built with `go 1.24` or older in `go.mod`, or without a CPU limit → GOMAXPROCS = node cores, throttling	`go 1.25`+ in `go.mod` and a CPU limit on every Go container, or set `GOMAXPROCS`
HPA scale-out storm after shrinking requests	`AverageValue` or request-rate metrics only
Unexpected night spike on a packed node	Shrink requests only; scale out on absolute metrics; placeholders (10.2)
Consolidation evicts the warm pod → cold start	2 replicas + PDB in the daytime; consolidation windows; startup boost
Tiny pods hit the pod-per-node limit (VCN-native)	Node sizes from section 9.5, extra VNIC profiles, or flannel on new clusters
Burstable node above baseline for long → all pods slow	Only low-CPU pools; watch the OCI `CpuUtilization` metric; or use the `kubeReserved` variant

Risk	Mitigation
Alpha and beta features (VPA `InPlace`, `CPUStartupBoost`, KEDA HTTP add-on)	Test in non-production; for production prefer `InPlaceOrRecreate`, kube-startup-cpu-boost, KEDA core scalers
Downscaler skips new workloads (15-minute grace period) and StatefulSets by default	Know the defaults; add `statefulsets` to the chart's `includedResources`
JVM thread pools sized for a boosted CPU limit	Boost requests only (`RequestsOnly`), modest factors, or `-XX:ActiveProcessorCount`
Lab ran containerd, OKE runs CRI-O	Repeat the in-place tests on an OKE development cluster before production

15. Open questions for the team

Answered on 27 October: runtimes (section 7), traffic pattern (section 10), what can go to zero (section 11) and cluster variants (section 12). Still open:

1. **Cold start per application.** Which frameworks (for example Express, NestJS or Next.js; Laravel or Symfony, on PHP-FPM or Octane/FrankenPHP)? How long does each take from `PodScheduled` to `Ready` (7.4)? This decides where true zero becomes acceptable.
2. **Traffic data.** After 1–2 weeks of Prometheus data, what does the query in 10.3 return per namespace?
3. **Cost per environment.** Once node pools are tagged (11.4), how much of the spend is non-production?
4. **Network requirements.** Do pods need VCN-routable IPs, NSG-based isolation, Zero Trust Packet Routing or virtual nodes? That decides the CNI, which cannot change after the cluster is created.
5. **Queues.** Will there be background workers, and on which broker (Redis, RabbitMQ, Kafka or OCI Streaming, OCI Queue)? That decides the KEDA scaler.
6. **Pricing.** Pay-as-you-go or a negotiated rate? Ratios stay the same, absolute savings change.

Appendix A. Files

Path	Content
`examples/01-app-deployment.yaml`	Burstable-QoS workload ready for in-place resize, PDB, opt-in labels
`examples/02-vpa-recommend-only.yaml`	VPA in `Off` mode
`examples/03-vpa-inplace.yaml`	VPA `InPlaceOrRecreate`, `RequestsOnly`, min/max
`examples/04-vpa-startup-boost.yaml`	Same as 03 plus CPU Startup Boost (alpha)
`examples/05-idle-shrinker-cronjob.yaml`	RBAC + script + CronJobs for off-hours in-place shrinking. Revision 2: containers are patched in pod order (the resize API rejects reordering), and one failing pod no longer stops the run
`examples/06-keda-cron-min-one.yaml`	KEDA ScaledObject: min 1, cron pre-warm, absolute CPU
`examples/07-hpa-absolute-cpu.yaml`	Plain HPA with `AverageValue`
`examples/08-karpenter-oci-burstable-and-preemptible.yaml`	KPO OCINodeClass/NodePool: burstable pool with consolidation windows, preemptible pool
`examples/09-oke-cluster-autoscaler-addon.json`	OKE Cluster Autoscaler add-on configuration

Path	Content
`examples/10-kube-startup-cpu-boost.yaml`	StartupCPUBoost resource
`examples/11a-runtime-nodejs.yaml`	Node.js Deployment: heap percentage, compile cache, probes, shrinker annotations
`examples/11b-runtime-php-fpm.yaml`	PHP-FPM + nginx Deployment with `pm = ondemand`
`examples/11c-runtime-go.yaml`	Go Deployment: CPU limit for GOMAXPROCS, explicit `GOMEMLIMIT`
`examples/12-nonprod-wake-on-request.yaml`	KEDA cron + HTTP add-on: zero at night, wakes on the first request
`examples/13-nonprod-namespace-downscaler.yaml`	GoKubeDownscaler namespace annotation, with an excluded workload
`examples/14-keda-queue-worker-zero.yaml`	Queue worker that scales 0 → N → 0 on a Redis list
`examples/15-capacity-placeholder.yaml`	PriorityClass + placeholder pods for spare node room
`lab/run-lab.sh`	Creates the kind cluster and reproduces the revision 1 **(lab)** results; `--delete` removes the cluster
`lab/run-runtime-lab.sh`	Reproduces the revision 2 results (runtimes, zero, placeholders); `--long` adds the idle-memory probes
`lab/runtime-*.yaml`, `lab/redis.yaml`	Probe pods used by `run-runtime-lab.sh`
`lab/cost-model.py`	Generates the cost tables in sections 6, 9 and 11

Appendix B. Sources

- OKE: supported versions, release notes, Kubernetes 1.35 on OKE, VPA on OKE, Cluster Autoscaler, CA add-on arguments, autoscaler best practices, KPO, KPO GA, KEDA + CA on OKE, virtual node resources, virtual vs managed nodes, preemptible nodes.
- OKE cluster variants and networking: basic vs enhanced, enhanced clusters, upgrading basic to enhanced, 20,000-node clusters, pod networking, VCN-native CNI, flannel CNI, multiple secondary VNICs, node tags, network load balancers.
- OCI Compute and pricing: burstable instances, burstable billing, preemptible instances, Ampere A2 OCPU definition, Flexible Network Load Balancer, price list API.
- Kubernetes: in-place resize, 1.35 GA blog, 1.36 pod-level resize beta, HPA scale-to-zero beta, node-pressure eviction.
- VPA: features, flags, known limitations, AEP-7862 CPU startup boost, VPA 1.8.0 release.
- Node.js: CLI options (`--max-old-space-size-percentage`), module compile cache, Node.js 20+ memory in containers (Red Hat), nodejs/node#65235, release index.
- PHP: PHP-FPM configuration, FrankenPHP worker mode, FrankenPHP performance, supported versions.
- Go: Go 1.25 release notes, container-aware GOMAXPROCS, runtime package (GOMEMLIMIT), downloads.
- Tools: KEDA, KEDA HTTP add-on, GoKubeDownscaler, kube-startup-cpu-boost, Kedify PodResourceProfile, EPCAS, OpenCost on OCI, Karpenter disruption.

Appendix C. Glossary of units

CPU and memory amounts in this report use Kubernetes notation, the same as the example manifests. Prices use OCI's billing units. Sources: Kubernetes resource units, OCI price list.

C.1 CPU

Term	Meaning	Example
CPU (Kubernetes)	One CPU in a request or limit is one virtual CPU (vCPU) of the node	`cpu: "1"` = one full vCPU
m (millicore)	One thousandth of a CPU; `1000m` = 1 CPU. Also called millicpu	`50m` = 0.05 CPU, 5% of one vCPU. `250m` = a quarter. `500m` and `0.5` are the same value: half a CPU
vCPU	One hardware thread. Linux and Kubernetes count it as one CPU	A node with 4 vCPUs fits pods whose requests add up to a little under `4` (the system reserves some)
OCPU	Oracle's billing unit for CPU: one physical core	x86 shapes (E4, E5, E6): 1 OCPU = 2 vCPUs. Ampere A1: 1 OCPU = 1 vCPU. AmpereOne A2: 1 OCPU = 2 single-threaded cores = 2 vCPUs
CPU request	CPU reserved for the container when the scheduler places it on a node. The cost model prices requests	30 pods × `50m` reserve 1.5 vCPUs
CPU limit	The most CPU the container may use, enforced in 100 ms periods	`1` = up to 100 ms of CPU time in every 100 ms. Above it the container is throttled (slowed down), not killed
Baseline (burstable nodes)	The share of each OCPU that an OCI burstable instance can use all the time. It is also the share you pay for	2 OCPUs with a 50% baseline are billed as 1 OCPU

C.2 Memory

Written as	Name	Size
`Ki`, `Mi`, `Gi`	kibibyte, mebibyte, gibibyte (binary units)	Powers of 1024. 1Mi = 1,048,576 bytes; 1Gi = 1024Mi = 1,073,741,824 bytes
`k`, `M`, `G`	kilobyte, megabyte, gigabyte (decimal units)	Powers of 1000. 1M = 1,000,000 bytes; 1G = 1,000,000,000 bytes
MiB, GiB	Same as `Mi`, `Gi`	The report writes measured values this way, for example "RSS 157 MiB"
MB, GB	Same as `M`, `G`	OCI prices memory per GB, and node sizes such as "8 GB" are OCI's figures

- `512Mi` is half a gibibyte (about 537 MB). `1Gi` is about 1.074 GB. `256M` is about 5% smaller than `256Mi`; both are valid in a manifest.
- **Case matters.** Uppercase `M` means megabytes, but lowercase `m` means milli: `400m` of memory is 0.4 *bytes*. Kubernetes accepts it without an error. As a request it reserves almost nothing; as a limit, the container cannot start. For memory, write `400Mi`.
- **Memory request:** memory reserved on the node when the pod is scheduled. A container may use more, but then it is among the first evicted when the node runs short (section 6.3).
- **Memory limit:** hard ceiling. Above it the container is OOM-killed (stopped by the kernel for using too much memory) and restarted.
- **RSS (resident set size):** the memory a process actually holds in RAM. The idle-memory measurements in sections 6.4 and 7 are RSS, or the container's total memory use (`memory.current`).

- **Heap:** the memory a language runtime manages for program objects (V8 in Node.js, the Go runtime). Its maximum is set by `--max-old-space-size` in Node.js and softly limited by `GOMEMLIMIT` in Go. It must stay below the container's memory limit.

C.3 Prices and time

Term	Meaning
USD, US$	US dollars at OCI's pay-as-you-go list prices on 27 October 2026, without discounts or taxes
OCPU-hour	Price of one OCPU for one hour. E5 Flex: US$0.030
vCPU-hour	The OCPU-hour price divided by the vCPUs in one OCPU. E5 Flex: US$0.030 / 2 = US$0.015. The report uses it because Kubernetes counts vCPUs
GB-hour	Price of 1 GB of memory for one hour. E5 Flex: US$0.002
Month	730 hours (365 days × 24 hours / 12 months). Every USD/month figure in this report is an hourly price × 730. Oracle's monthly caps and free allowances use 744 hours (31 days); for example, the enhanced-cluster fee is capped at US$74.40
24x7	Running every hour of the month: 730 hours
Office hours, off-hours	Office hours are Monday to Friday, 08:00–20:00 São Paulo time: 60 of the 168 hours in a week (36%). Off-hours are the other 108 hours (64%)
Cost of a pod	The cost of the node capacity its *requests* reserve, not of what it actually uses. This assumes the node autoscaler keeps nodes well packed (section 2)

C.4 Worked example: what one warm pod costs

A pod requesting `50m` CPU and `1Gi` memory on E5 Flex nodes, running 24x7:

Part	Calculation	USD/month
CPU	`50m` = 0.05 vCPU. 0.05 × US$0.015 per vCPU-hour × 730 hours	0.55
Memory	`1Gi` = 1.074 GB. 1.074 × US$0.002 per GB-hour × 730 hours	1.57
Total		2.12

- With the CPU request at `500m`, the CPU part is 10 times larger: 0.5 × 0.015 × 730 = US$5.47, and the pod costs US$7.04 a month. CPU is then 78% of the cost (5.47 / 7.04), the figure used in sections 1 and 6.2.
- The "7.5 GB of RAM cost as much as 1 vCPU" figure in section 6.1 is US$0.015 / US$0.002.
- The 30-service scenarios in section 6.5 apply the same arithmetic per hour of the week, then multiply by the number of pods and services.
- **Memory unit assumption:** OCI's price list says "gigabyte", so the cost model converts with 1Gi = 1.074 GB. If OCI actually counts memory in binary gigabytes (some Oracle price lists, such as Compute Cloud@Customer, say "gibibyte"), the memory figures are about 7% lower. The conclusions do not change: CPU would be an even larger share of the cost.

C.5 Other terms used with numbers

Term	Meaning
p90, p95	90th and 95th percentile: 90% (or 95%) of the measured values are at or below this value. VPA bases CPU requests on the p90 of usage
Replica	One copy of a pod. "3 replicas" means three identical pods sharing the traffic
Off-hours share	The fraction of a week's CPU use that falls outside office hours (section 10.3). 0.64 means use is flat; lower values mean use concentrates in office hours
`Utilization` and `AverageValue`	Two ways to set an HPA CPU target: as a percentage of the request (`Utilization`), or as an absolute amount such as `400m` (`AverageValue`). Section 5.5 explains why this report uses the absolute form